

Continuous Integration with Jenkins

Creating a Build Pipeline: Jenkins + GitHub + Maven

A Step-by-Step Proof of Concept (POC) Guide

Document Overview

This document walks through the practical steps of setting up Continuous Integration (CI) using Jenkins, connected to a GitHub repository, with Maven as the build tool. Each step is illustrated with an actual screenshot taken during the setup, followed by a detailed explanation of what was done and why.

Scope

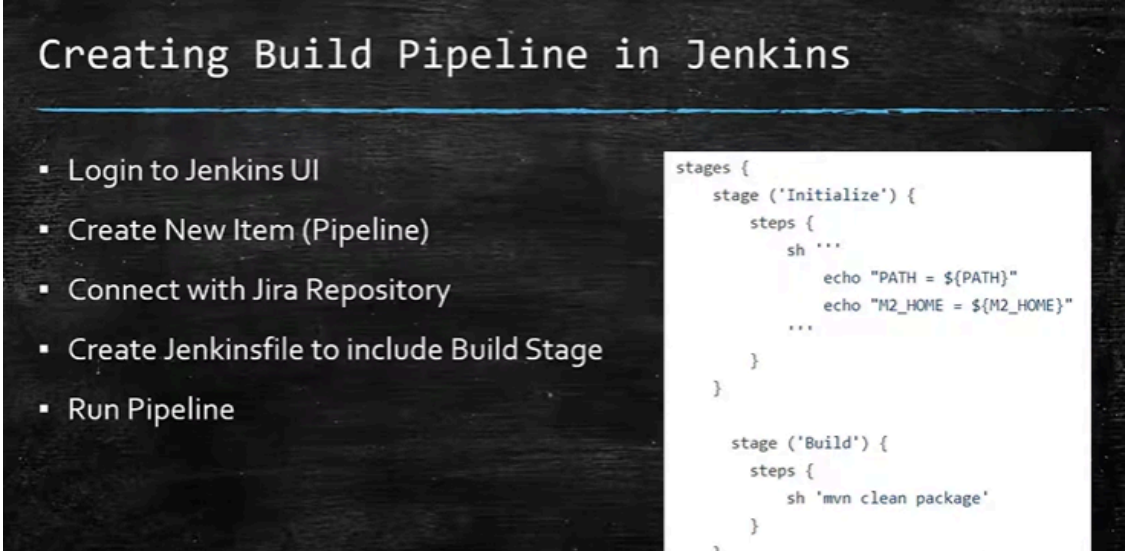
- Creating a new Pipeline job in Jenkins
- Configuring the job to connect with a GitHub repository
- Writing and committing a Jenkinsfile that defines the build stages
- Running the pipeline and diagnosing a build failure
- Fixing the Jenkins Tools configuration
- Re-running the pipeline to a successful build

Table of Contents

- 1. Reference Slide — Creating Build Pipeline in Jenkins
- 2. Step 1 — Start a New Item in Jenkins
- 3. Step 2 — Name the Job and Select 'Pipeline' Type
- 4. Step 3 — General Configuration and Build Retention
- 5. Step 4 — Link the GitHub Project and Configure Triggers
- 6. Step 5 — Configure the Pipeline Definition (Pipeline script from SCM)
- 7. Step 6 — Locate the Repository on GitHub
- 8. Step 7 — Create the Jenkinsfile in the Repository
- 9. Step 8 — First Build Attempt and Failure Diagnosis
- 10. Step 9 — Root Cause: Missing Jenkinsfile Case Sensitivity
- 11. Step 10 — Fixing Global Tool Configuration (Git)
- 12. Step 11 — Triggering the Build Again
- 13. Step 12 — Successful Build Confirmation
- 14. Step 13 — Reviewing the Full Pipeline Run in Blue Ocean
- 15. Summary and Key Takeaways

1. Reference Slide — Creating Build Pipeline in Jenkins

The following reference slide outlines the standard five-step process for creating a build pipeline in Jenkins, which this POC follows: logging in to the Jenkins UI, creating a new Pipeline item, connecting to a source code repository, creating a Jenkinsfile that defines the Build stage, and finally running the pipeline. The adjacent code snippet shows the Jenkinsfile structure used, with an 'Initialize' stage that prints environment variables and a 'Build' stage that runs the Maven package command.



Creating Build Pipeline in Jenkins

- Login to Jenkins UI
- Create New Item (Pipeline)
- Connect with Jira Repository
- Create Jenkinsfile to include Build Stage
- Run Pipeline

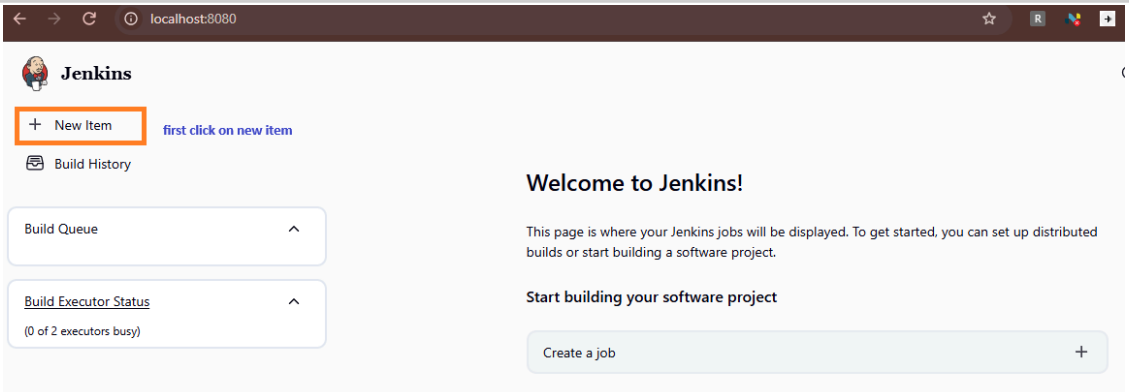
```
stages {
  stage ('Initialize') {
    steps {
      sh '''
        echo "PATH = ${PATH}"
        echo "M2_HOME = ${M2_HOME}"
      '''
    }
  }

  stage ('Build') {
    steps {
      sh 'mvn clean package'
    }
  }
}
```

Reference slide: standard steps for creating a build pipeline in Jenkins, alongside the Jenkinsfile stage structure.

2. Step 1 — Start a New Item in Jenkins

After logging in to the Jenkins dashboard at <http://localhost:8080> (accessed through an SSH tunnel), the first action is to click on New Item in the left-hand navigation panel. This is the entry point for creating any job in Jenkins, whether it is a Freestyle project, a Pipeline, or another job type.

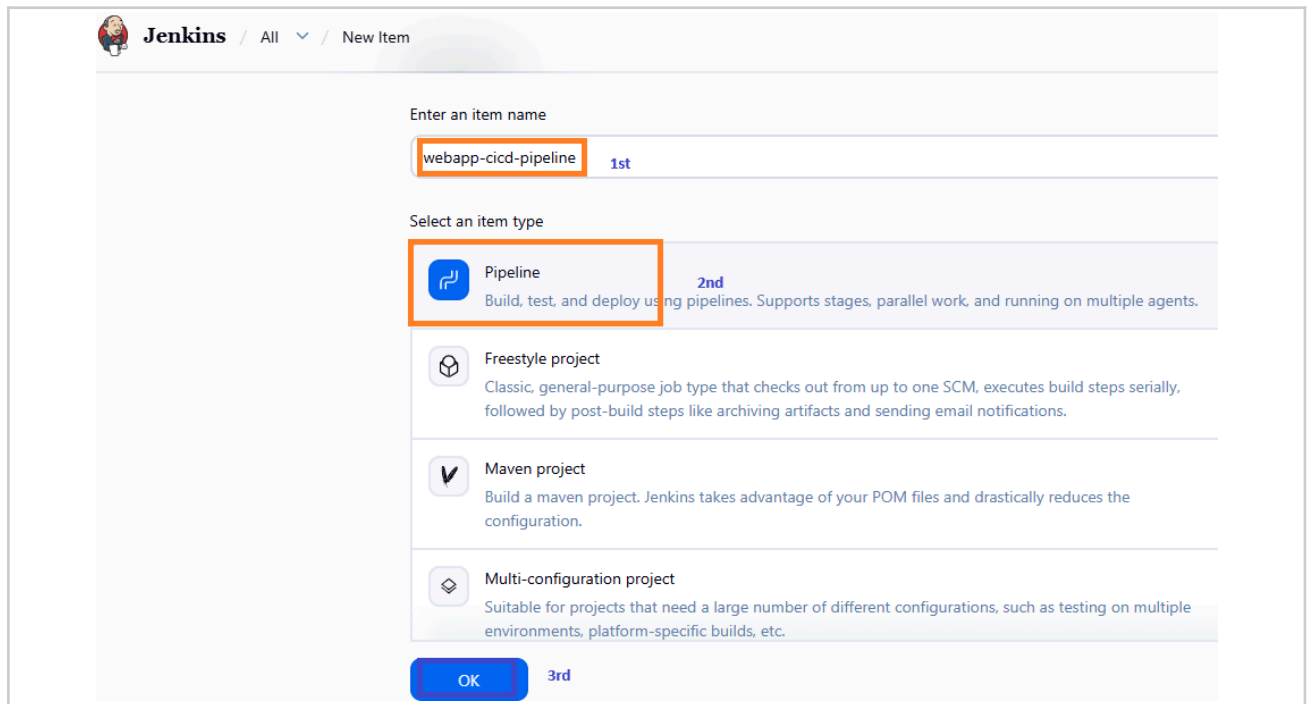


The screenshot shows the Jenkins dashboard at localhost:8080. The 'New Item' button in the left-hand navigation panel is highlighted with an orange box. The main content area displays 'Welcome to Jenkins!' and a 'Create a job' button.

The Jenkins dashboard homepage. 'New Item' is highlighted as the starting point for creating a pipeline job.

3. Step 2 — Name the Job and Select 'Pipeline' Type

On the New Item screen, a name is entered for the job — in this case, webapp-cicd-pipeline. Jenkins then presents several item types to choose from: Pipeline, Freestyle project, Maven project, and Multi-configuration project. Pipeline is selected because it allows the entire build process to be defined as code (a Jenkinsfile), supports multiple stages, and can run across different agents. Clicking OK creates the job and proceeds to its configuration screen.

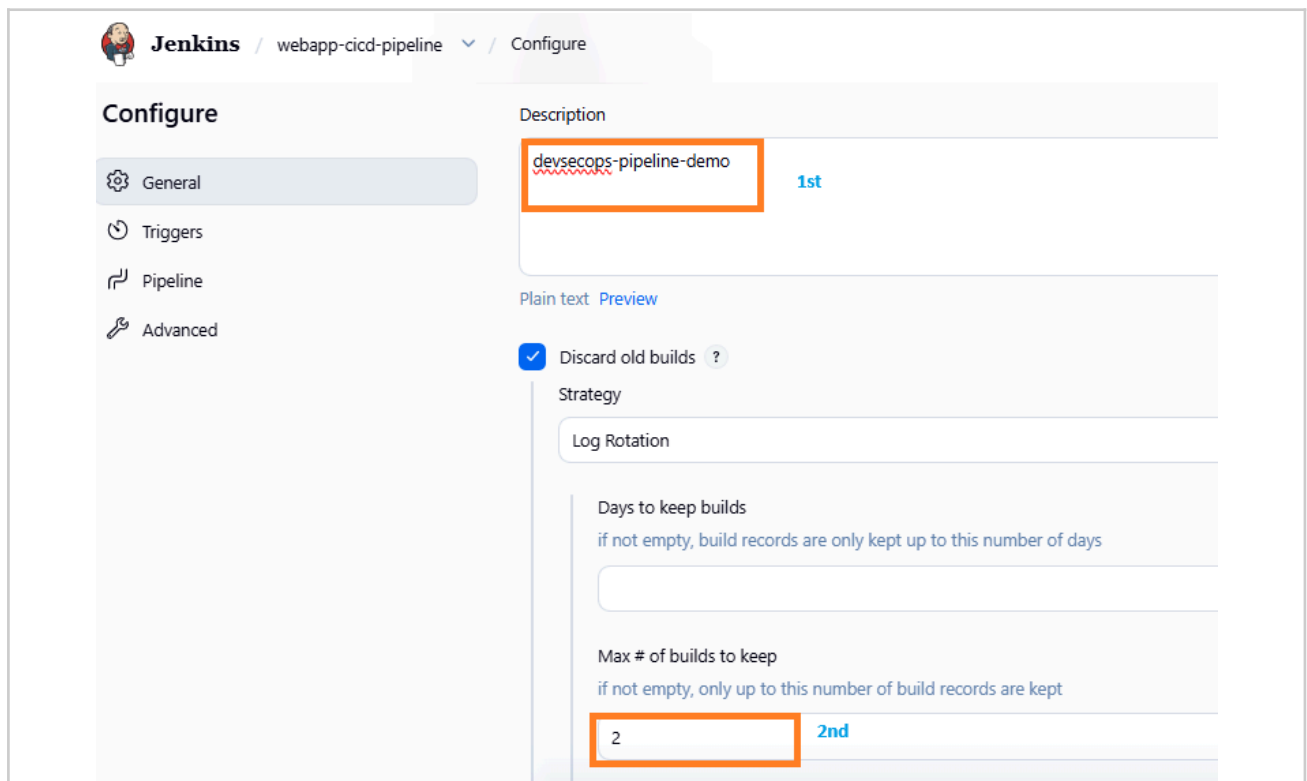


The screenshot shows the Jenkins 'New Item' configuration page. At the top, the breadcrumb is 'Jenkins / All / New Item'. The first section, 'Enter an item name', has a text input field containing 'webapp-cicd-pipeline' and a '1st' label. The second section, 'Select an item type', lists four options: 'Pipeline' (highlighted with an orange box and a '2nd' label), 'Freestyle project', 'Maven project', and 'Multi-configuration project'. At the bottom, there is an 'OK' button and a '3rd' label.

Naming the new item 'webapp-cicd-pipeline' and selecting the Pipeline project type.

4. Step 3 — General Configuration and Build Retention

The job configuration screen opens with the General tab active. A short description, `devsecops-pipeline-demo`, is added for documentation purposes. The Discard old builds option is enabled with a Log Rotation strategy, and Max # of builds to keep is set to 2. This prevents the Jenkins workspace and build history from growing indefinitely and consuming disk space on the t3.micro instance, which has limited storage — a precaution that proved directly relevant later when disk space issues were encountered during pipeline execution.



The screenshot shows the Jenkins 'Configure' page for the 'webapp-cicd-pipeline' job. The breadcrumb is 'Jenkins / webapp-cicd-pipeline / Configure'. On the left, the 'Configure' sidebar has tabs for 'General' (selected), 'Triggers', 'Pipeline', and 'Advanced'. The main area shows the 'Description' field with the text 'devsecops-pipeline-demo' and a '1st' label. Below this, the 'Plain text' tab is selected. The 'Discard old builds' checkbox is checked. Under the 'Strategy' section, 'Log Rotation' is selected. The 'Days to keep builds' field is empty. The 'Max # of builds to keep' field is set to '2' (highlighted with an orange box) and has a '2nd' label.

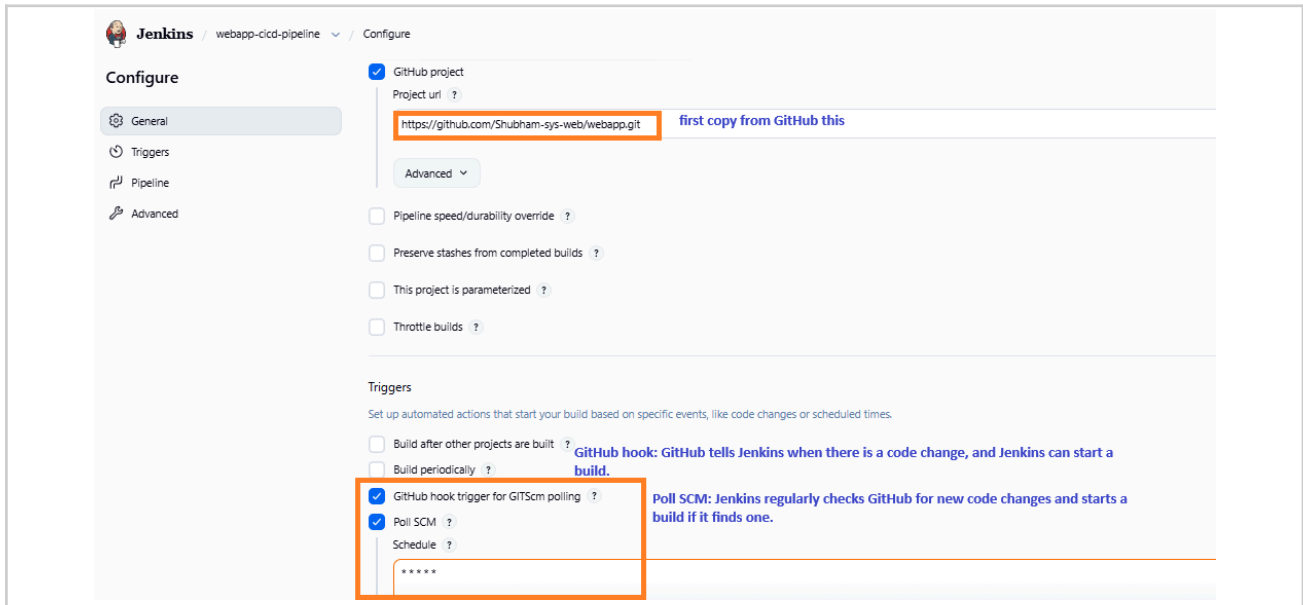
General configuration tab: description added and build retention set to keep only the last 2 builds.

5. Step 4 — Link the GitHub Project and Configure Triggers

Scrolling down, the GitHub project checkbox is enabled and the Project URL is set to the repository being built: `https://github.com/Shubham-sys-web/webapp.git`. This associates the job with the GitHub project for display purposes and enables GitHub-specific features.

In the Triggers section, two options are enabled:

- GitHub hook trigger for GITScm polling — allows GitHub to notify Jenkins immediately when new code is pushed, so a build can start automatically (via a webhook).
- Poll SCM — as a fallback, Jenkins periodically checks the repository itself for new commits and triggers a build if changes are found, even if the webhook notification does not arrive.



GitHub project URL configured, with both GitHub hook trigger and Poll SCM enabled for automatic build triggering.

6. Step 5 — Configure the Pipeline Definition (Pipeline script from SCM)

In the Pipeline section, the Definition dropdown is set to Pipeline script from SCM. This tells Jenkins that the pipeline's instructions (the Jenkinsfile) live inside the source code repository itself, rather than being written directly into the Jenkins job configuration. This is the recommended approach because it keeps the pipeline definition version-controlled alongside the application code.

The SCM is set to Git, and the same Repository URL (`https://github.com/Shubham-sys-web/webapp.git`) is entered again here. Since the repository is public, Credentials is left as - none -. The Branch Specifier is left at its default value, `*/master`, which tells Jenkins to build from the master branch. After these settings are confirmed, Apply and then Save are clicked to store the configuration.

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM 1st

SCM ?

Git 2nd

Repositories ?

Repository URL ?

https://github.com/Shubham-sys-web/webapp.git 3rd copy from github

Credentials ?

- none -

Advanced

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

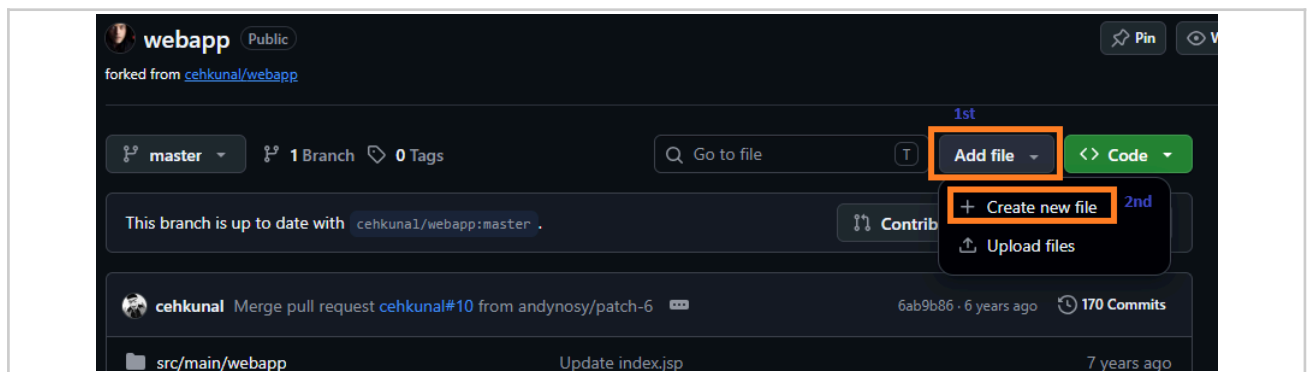
*/master that is by default

Save Apply leave it all by default and click on apply and save

Pipeline definition set to 'Pipeline script from SCM', pointing to the Git repository and the master branch.

7. Step 6 — Locate the Repository on GitHub

Switching to the GitHub side, the webapp repository (a fork of cehkunal/webapp) is opened. This is where the Jenkinsfile needs to be created, since the Jenkins job was just configured to pull its pipeline definition from this repository's source control. The Add file dropdown is used, and Create new file is selected.



The webapp GitHub repository, with 'Add file' → 'Create new file' selected to add the Jenkinsfile.

8. Step 7 — Create the Jenkinsfile in the Repository

A new file is created directly in the GitHub web editor. The file's content defines the pipeline structure using Declarative Pipeline syntax:

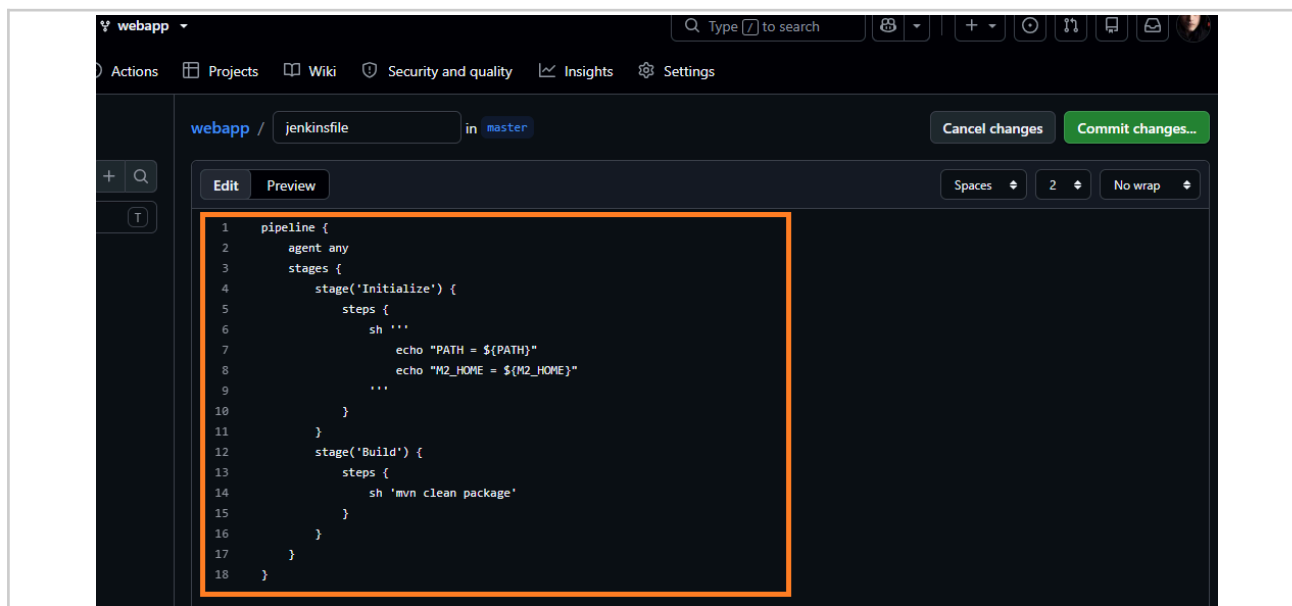
```
pipeline {
  agent any
  stages {
    stage('Initialize') {
      steps {
        sh '''
          echo "PATH = ${PATH}"
          echo "M2_HOME = ${M2_HOME}"
        '''
      }
    }
  }
}
```

```

    }
    stage('Build') {
        steps {
            sh 'mvn clean package'
        }
    }
}

```

This defines two stages: Initialize, which prints the current PATH and M2_HOME environment variables for diagnostic purposes, and Build, which runs mvn clean package — the standard Maven command that compiles the source code, runs tests, and packages the application into a deployable artifact (a .war file, since this is a web application). The change is committed directly to the master branch using the Commit changes button.



Creating the Jenkinsfile in the GitHub web editor with the Initialize and Build stages, ready to commit.

9. Step 8 — First Build Attempt and Failure Diagnosis

Back in Jenkins, build #1 was triggered automatically by the SCM change (the Jenkinsfile commit). However, the build status shows a red cross icon next to #1 (Sep 11, 2026, 7:16:09 AM), indicating failure. To investigate, Console Output is opened from the left-hand menu — this is the standard first troubleshooting step for any failed Jenkins build, as it shows the exact log output produced during the run.



Build #1 failed (red cross icon). Console Output is selected to view the failure details.

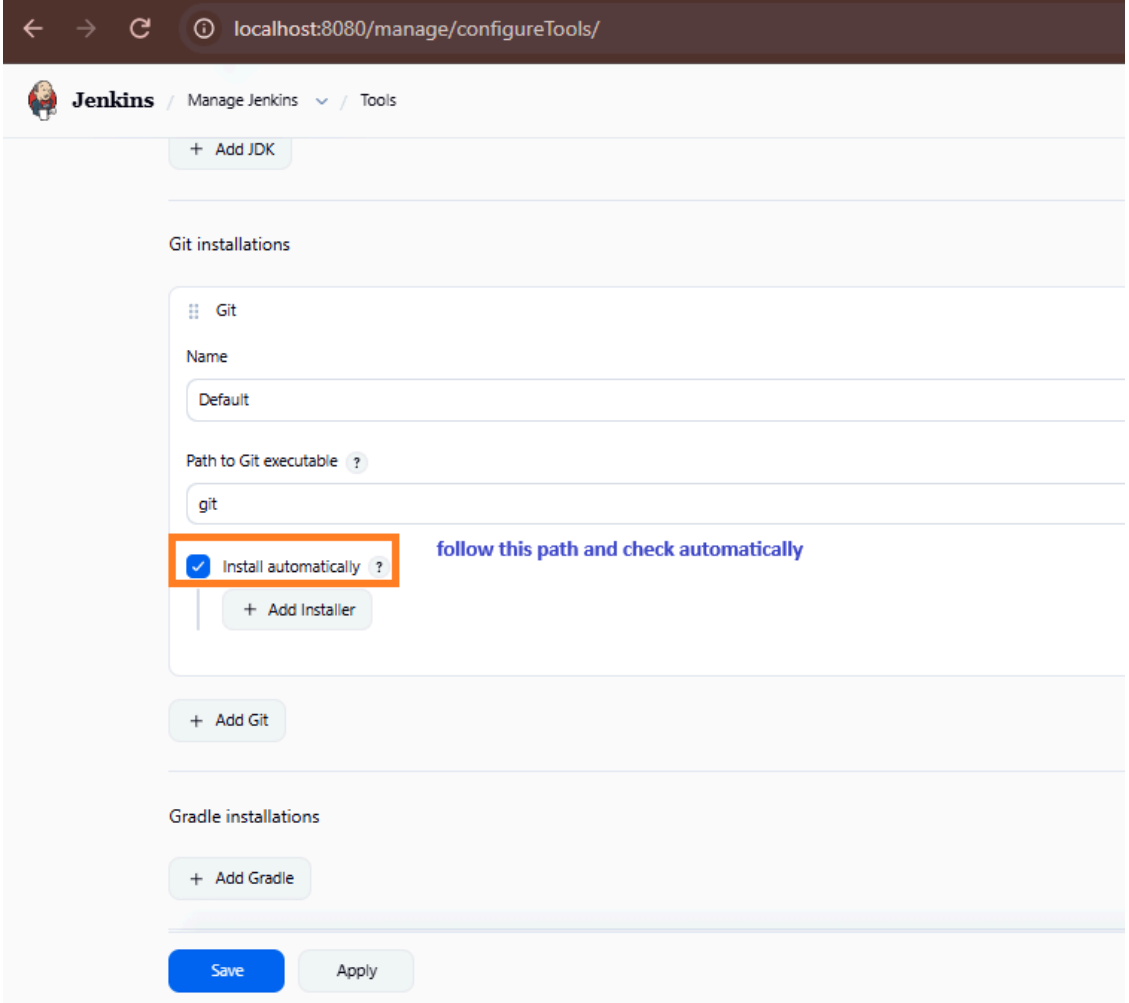
10. Step 9 — Root Cause: Missing Jenkinsfile Case Sensitivity

The console output reveals the exact error:

```
Started by an SCM change
ERROR: Unable to find Jenkinsfile from git https://github.com/Shubham-sys-web/webapp.git
Finished: FAILURE
```

Jenkins, by default, looks for a file named exactly Jenkinsfile (capital J) in the repository root. Investigation of the repository showed the file had been committed as jenkinsfile (lowercase j). Since Git and Linux filesystems are case-sensitive, Jenkins could not locate the file under the name it expected, even though a file with a similar name existed.

Note: The fix applied was to rename the file to the exact expected case, Jenkinsfile, directly on GitHub. Alternatively, the Script Path field in the job's Pipeline configuration could have been changed to match the lowercase filename instead — renaming the file was chosen as it follows the standard Jenkins convention.



Console output showing the 'Unable to find Jenkinsfile' error — caused by a filename case mismatch.

11. Step 10 — Fixing Global Tool Configuration (Git)

While investigating the build environment, the Git tool configuration was also reviewed under Manage Jenkins → Tools (accessible at /manage/configureTools/). Under Git installations, the Install automatically option is enabled, which allows Jenkins to automatically download and manage the Git executable rather than depending on a specific pre-installed path. This ensures the pipeline's Git operations (cloning the repository, checking out branches) work reliably regardless of what is or is not already installed on the underlying instance.



Build Now

1st click here



Delete Pipeline



Configure



Rename



GitHub



Favorite



Open Blue Ocean



Stages



Pipeline Syntax



GitHub Hook Log



Git Polling Log

Builds >



Filter



it is working right now

Today

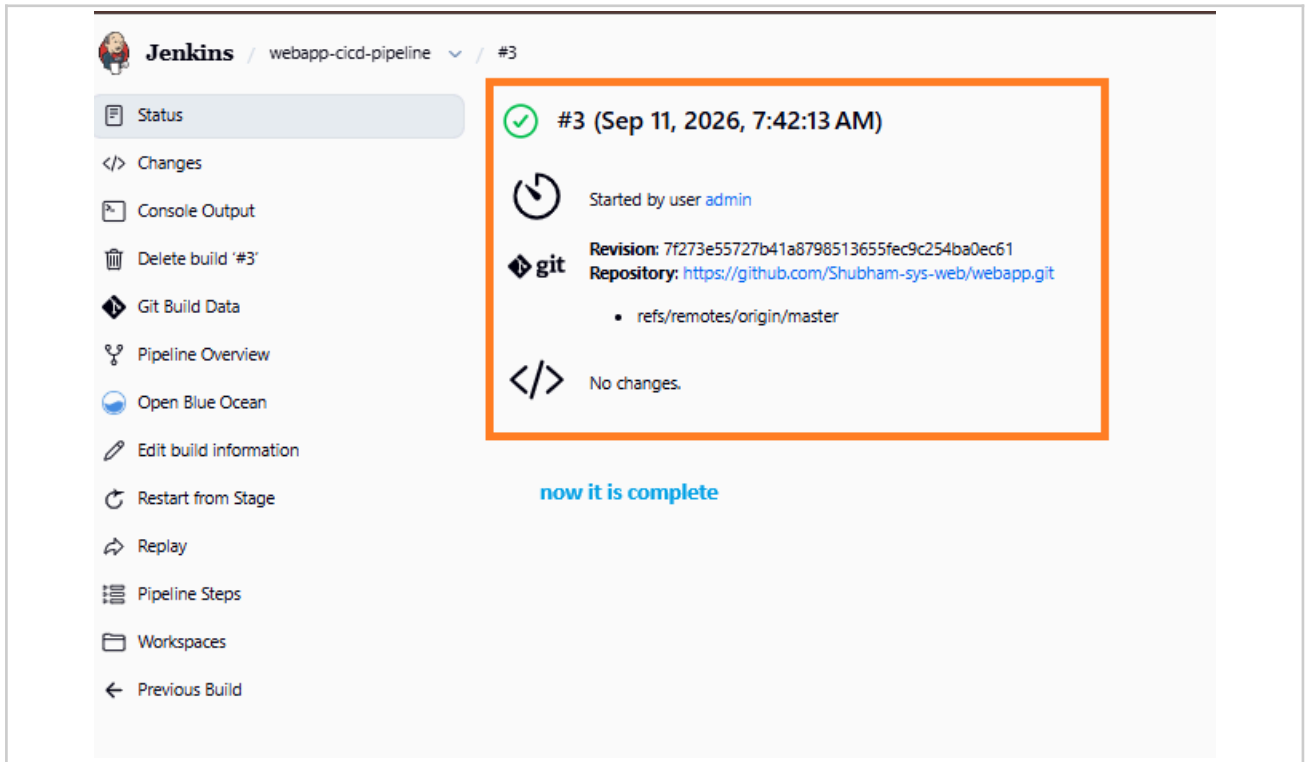


#3 7:42 AM



12. Step 11 — Triggering the Build Again

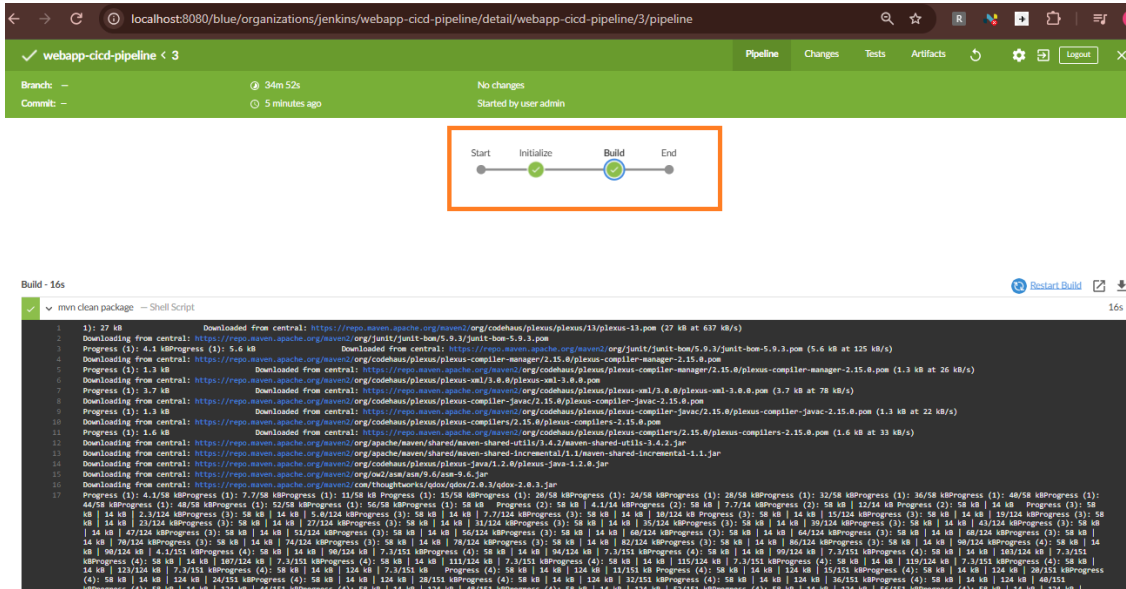
With the Jenkinsfile renamed correctly, the pipeline job page is opened again and Build Now is clicked from the left-hand menu to manually trigger a new run. The Builds panel at the bottom shows the history: build #1 and #2 both failed (red icons), while build #3, just started at 7:42 AM, is shown as actively running (red progress bar), confirming that this time the pipeline is executing rather than failing immediately.



Manually triggering build #3 with 'Build Now'. The build is shown actively running, unlike the earlier failed attempts.

13. Step 12 — Successful Build Confirmation

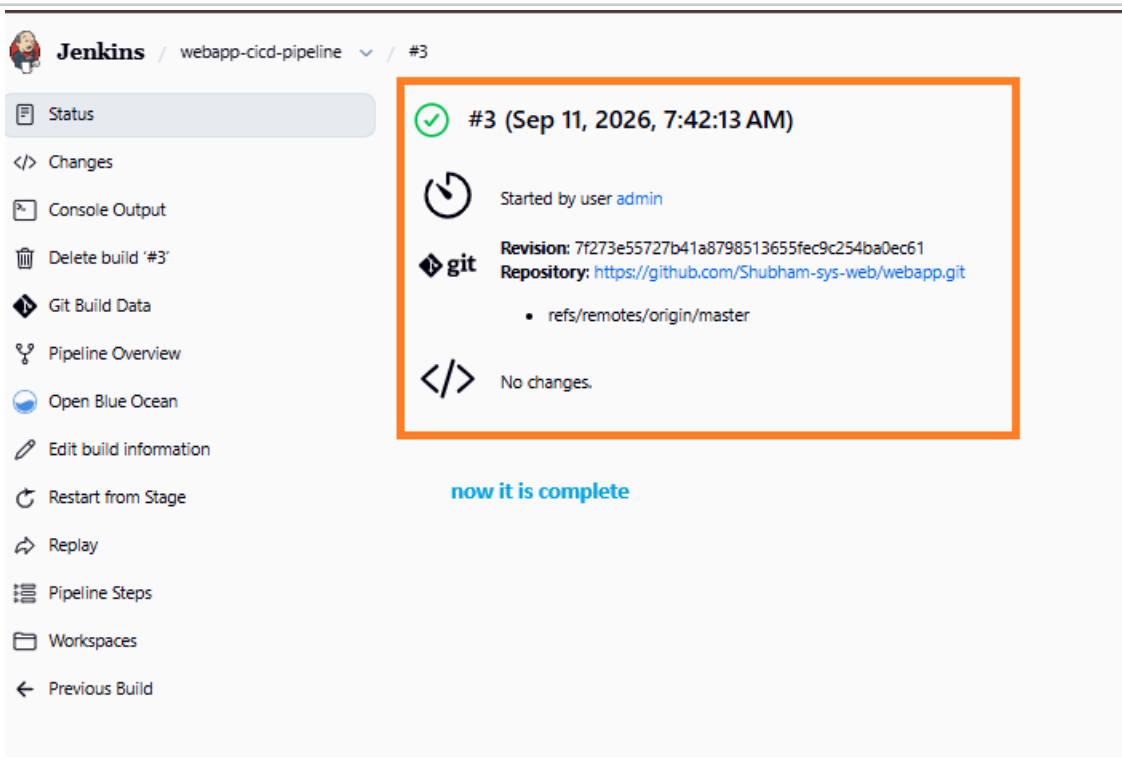
Once build #3 completes, its status page confirms success with a green checkmark: #3 (Sep 11, 2026, 7:42:13 AM), started by user admin. The Git Build Data section confirms the exact commit that was built (revision 7f273e55727b41a8798513655fec9c254ba0ec61) and the branch reference (refs/remotes/origin/master), with No changes noted for this particular run since it was manually triggered rather than by a new commit.



Build #3 completed successfully (green checkmark), with Git revision and repository details confirmed.

14. Step 13 — Reviewing the Full Pipeline Run in Blue Ocean

Opening the build in Blue Ocean (Jenkins' modern visual pipeline interface) provides a clearer, graphical view of the pipeline's stages: Start → Initialize → Build → End, all marked with green checkmarks. The Build stage log below shows Maven downloading its dependencies from Maven Central (repo.maven.apache.org) — plugin POMs and JARs required to compile and package the project — followed by the packaging of the WAR file and a final BUILD SUCCESS message. The total run took 34 minutes on this occasion (a first-time run against a t3.micro instance, before Maven's local dependency cache was warm); subsequent builds are expected to complete significantly faster since dependencies are cached locally after the first successful download.



Blue Ocean pipeline view showing the four stages of the run — Start, Initialize, Build, End — each completed successfully.

15. Summary and Key Takeaways

This POC demonstrates a complete, working Continuous Integration setup using Jenkins Pipeline as Code, sourced directly from a GitHub repository. The key steps performed were:

1. Created a Pipeline-type job in Jenkins and linked it to a GitHub repository via GitHub project settings and SCM polling/webhook triggers.
2. Configured the pipeline definition as 'Pipeline script from SCM', keeping the build logic version-controlled with the application code rather than hardcoded into the Jenkins UI.
3. Authored a Jenkinsfile with two stages — Initialize (environment diagnostics) and Build (Maven packaging) — and committed it to the repository.
4. Diagnosed and resolved a build failure caused by filename case sensitivity (jenkinsfile vs. Jenkinsfile), using Jenkins' Console Output as the primary debugging tool.
5. Verified the Git tool configuration in Manage Jenkins → Tools to ensure reliable source control operations.
6. Re-triggered and confirmed a successful build, validating the pipeline end-to-end from source checkout through to a packaged WebApp.war artifact.

Note: A recurring lesson from this exercise is that Jenkins error messages and Console Output logs are precise and should be read carefully — nearly every failure encountered (missing Jenkinsfile, GPG key mismatch, insufficient Java version, malformed XML, low disk space) was diagnosable directly from the log text, without needing external troubleshooting.